Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления
Кафедра информационных технологий автоматизированных систем

ОТЧЕТ
по контрольной работе № 1

по дисциплине
«ИТиВП»

на тему «Серверная разработка»

Выполнил: студент группы 320601
А. С. Образцов

Проверила: Н. В. Хаджинова

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Целью контрольной работы является разработка *REST API* на платформе *Node.js* с аутентификацией по *JWT*, хранением данных в *PostgreSQL* и полным набором *CRUD*”=операций для сущности, соответствующей теме курсового проекта. Тема курсового проекта — сервис планирования задач по методу *GTD* (*Getting Things Done*) с контекстами и напоминаниями (вариант 27).

Метод *GTD* предполагает, что все дела человека сначала собираются во входящие, а затем разбираются по спискам. Поэтому основной сущностью сервиса выбрана задача (*Task*): она хранит заголовок, заметки, статус (*inbox*, *next*, *waiting*, *scheduled*, *someday*, *done*), срок выполнения *dueDate*, время напоминания *remindAt*, контекст *contextId* и владельца *ownerId*. Помимо стандартных полей *id*, *createdAt* и *updatedAt* сущность содержит семь собственных полей, что превышает минимум в четыре поля. Поле *status* ограничено перечислением, поле *title* и обе даты проходят валидацию, а уникальность обеспечивается полем *email* модели пользователя.

2 ХОД РАБОТЫ

2.1 Выбор технологий и структура проекта

Сервис написан на *Node.js* с фреймворком *Express 5*. Данные хранятся в *PostgreSQL 16*, запущенной в контейнере *Docker*, доступ к базе выполняется через *ORM Sequelize*. Пароли хешируются библиотекой *bcrypt*, токены подписываются библиотекой *jsonwebtoken*, входные данные проверяются схемами *Joi*. Для защиты используются *helmet*, *cors* и *express-rate-limit*. Выбор *PostgreSQL* вместо *MongoDB* объясняется структурой данных: у задач, контекстов и пользователей есть чёткие связи и ограничения целостности, которые реляционная база проверяет сама.

Код разделён на слои: маршруты (*routes*) описывают цепочку *middleware*, контроллеры (*controllers*) читают проверенные данные запроса и вызывают сервисы (*services*), в которых сосредоточены бизнес-логика и проверка прав доступа, а модели (*models*) описывают таблицы и связи. Настройки хранятся в файле *.env*, образец *.env.example* лежит в репозитории.

Исходный код размещён в репозитории *https://github.com/Alexandr-Obraztsov/gtd-task-service*, ветка *kr21*. Коллекция *Postman* находится в файле *postman/gtd-secure-api.postman_collection.json*, документация — в файлах *API.md* и *docs/openapi.json*, инструкция по запуску — в *README.md*.

2.2 Схема базы данных

База данных состоит из четырёх таблиц. Таблица *users* хранит учётные записи: уникальный *email*, хеш пароля, роль (*user*, *moderator*, *admin*), счётчик неудачных входов и время окончания блокировки. Таблица *refresh_tokens* хранит хеши выданных *refresh*-токенов. Таблицы *contexts* и *tasks* хранят контексты и задачи, у каждой записи есть владелец *ownerId*. Схема базы данных представлена на рисунке 2.1.

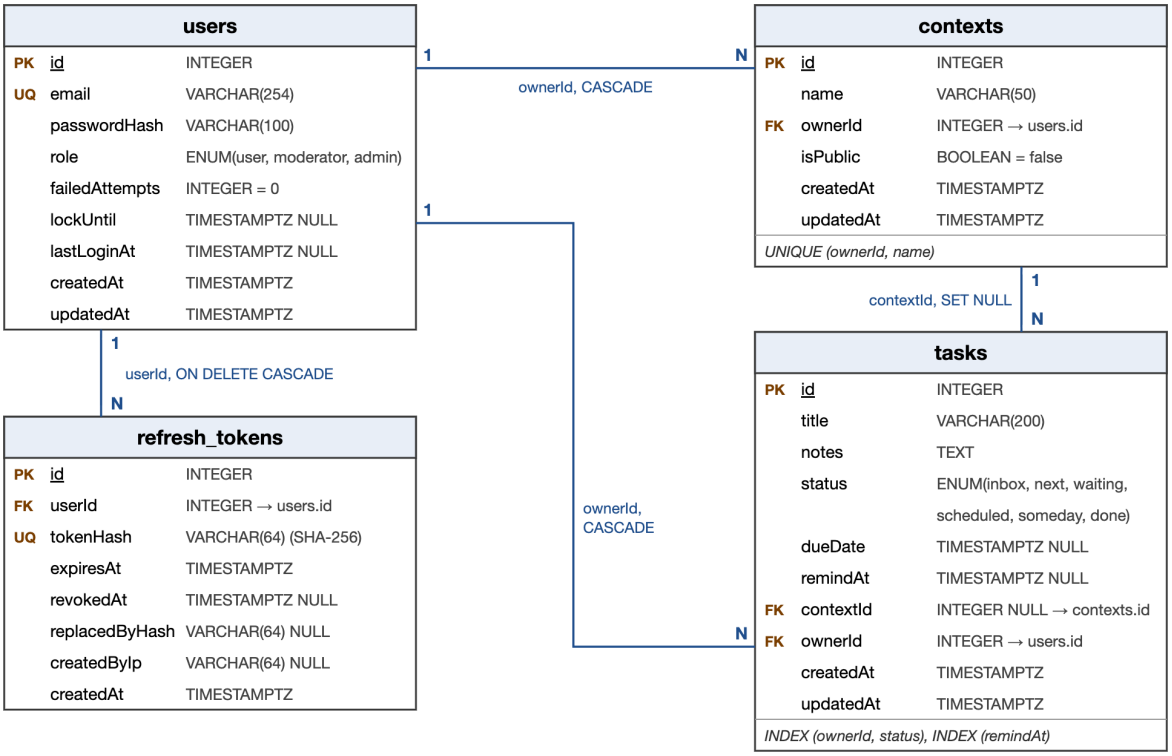


Рисунок 2.1 – Схема базы данных сервиса

При удалении пользователя каскадно удаляются его задачи, контексты и токены. При удалении контекста задачи не удаляются: поле *contextId* у них становится пустым.

2.3 Модели

Модель пользователя содержит уникальный *email* с проверкой формата и роль из перечисления. Метод *toJSON* возвращает только открытые поля, поэтому хеш пароля не может попасть в ответ даже по ошибке в контроллере:

```
const PUBLIC_FIELDS = Object.freeze(
  ['id', 'email', 'role', 'createdAt', 'updatedAt']);

class User extends Model {
  toJSON() {
    const values = this.get({ plain: true });
    return Object.fromEntries(
      PUBLIC_FIELDS.map((field) => [field, values[field]]));
  }
}

User.init({
  id: { type: DataTypes.INTEGER, autoIncrement: true,
```

```
    primaryKey: true },
  email: { type: DataTypes.STRING(254), allowNull: false,
    unique: true, validate: { isEmail: true } },
  passwordHash: { type: DataTypes.STRING(100),
    allowNull: false },
  role: { type: DataTypes.ENUM(...ROLE_VALUES),
    allowNull: false, defaultValue: ROLES.USER },
}, { sequelize, modelName: 'User', tableName: 'users' });
```

Модель задачи ограничивает статус перечислением *GTD* и задаёт индексы для выборки по владельцу и статусу и для поиска наступивших напоминаний:

```
Task.init({
  id: { type: DataTypes.INTEGER, autoIncrement: true,
    primaryKey: true },
  title: { type: DataTypes.STRING(200), allowNull: false },
  notes: { type: DataTypes.TEXT, allowNull: false,
    defaultValue: '' },
  status: { type: DataTypes.ENUM(...TASK_STATUS_VALUES),
    allowNull: false, defaultValue: TASK_STATUS.INBOX },
  dueDate: { type: DataTypes.DATE, allowNull: true },
  remindAt: { type: DataTypes.DATE, allowNull: true },
  contextId: { type: DataTypes.INTEGER, allowNull: true },
  ownerId: { type: DataTypes.INTEGER, allowNull: false },
}, {
  sequelize, modelName: 'Task', tableName: 'tasks',
  indexes: [{ fields: ['ownerId', 'status'] },
    { fields: ['remindAt'] }],
});
```

2.4 Маршруты, роли и права доступа

Маршруты задач требуют аутентификации для всех операций, а каждый маршрут имеет собственную схему проверки входных данных. Маршрут *GET /tasks* принимает необязательные фильтры *status*, *contextId* и *dueBefore*, а также постраничный вывод — это дополнительный эндпоинт с фильтрацией, которого требует задание. Для *GET /tasks/reminders/due* возвращаются задачи пользователя, время напоминания которых уже наступило:

```
const router = Router();

router.use(authenticate);
```

```
router.get('/', validate(schemas.list), controller.list);
router.post('/', validate(schemas.create), controller.create);
router.get('/reminders/due', validate(schemas.dueReminders),
  controller.dueReminders);
router.get('/:id', validate(schemas.getOne), controller.getOne);
router.put('/:id', validate(schemas.replace),
  controller.replace);
router.patch('/:id', validate(schemas.update),
  controller.update);
router.delete('/:id', validate(schemas.remove),
  controller.remove);
```

Права доступа заданы заданием контрольной работы: создавать задачи может любая аутентифицированная роль (*user* и выше), обновлять — автор задачи или администратор, удалять — только администратор. Проверка выполняется в сервисе, поскольку для неё нужна сама запись из базы данных. Администратору разрешено обновление, остальные роли обновляют только свои задачи. Удаление проверяет роль до обращения к базе, поэтому ответ не раскрывает существование чужой задачи:

```
const TASK_ACCESS_POLICY = Object.freeze({
  ...PRIVILEGED_ROLE_BY_ACTION,
  [ACTIONS.UPDATE]: ROLES.ADMIN,
});

function replaceTask(actor, id, input) {
  return updateTask(actor, id,
    { ...FULL_UPDATE_DEFAULTS, ...input });
}

async function deleteTask(actor, id) {
  if (!hasRole(actor.role, ROLES.ADMIN)) {
    throw new ForbiddenError(
      'Удалять задачи может только администратор',
      'ADMIN_ONLY');
  }
  const task = await loadTask(actor, id, ACTIONS.DELETE);
  await task.destroy();
}
```

Метод *PUT* выполняет полную замену: обязателен только заголовок, а не переданные поля (*notes*, *status*, *dueDate*, *remindAt*, *contextId*) сбрасываются к значениям по умолчанию. Метод *PATCH* меняет только переданные поля.

2.5 Валидация входных данных

Для каждого источника данных каждого маршрута (*params*, *query*, *body*) описана схема *Joi*, неизвестные поля запрещены. Схема создания задачи требует заголовок, ограничивает его длину, проверяет статус по перечислению и приводит даты к формату *ISO 8601* в допустимом диапазоне; схема замены повторяет её:

```
const title = Joi.string().trim().min(1).max(200);
const notes = Joi.string().trim().max(2000).allow('');
const status = Joi.string().valid(...TASK_STATUS_VALUES);
const boundedDate = Joi.date().iso()
  .min('2000-01-01T00:00:00Z').max('2100-12-31T23:59:59Z');

const create = {
  body: Joi.object({
    title: title.required(),
    notes,
    status,
    dueDate: boundedDate.allow(null),
    remindAt: boundedDate.allow(null),
    contextId: id.allow(null),
  }),
};

const replace = { params: idParams, body: create.body };
```

Для регистрации пароль должен содержать не менее восьми символов, букву, цифру и спецсимвол, а *email* приводится к нижнему регистру и проверяется на формат. Кроме схем, действует ограничение тела запроса размером 10 КБ и приём только формата *application/json*.

2.6 Аутентификация

При регистрации пароль хешируется алгоритмом *bcrypt* с 12 раундами, в базу попадает только хеш. Уникальность *email* проверяется запросом к базе и дополнительно гарантируется ограничением *UNIQUE* на столбец; при повторной регистрации сервер отвечает статусом 409:

```
async function registerUser({ email, password }) {
  const existing = await User.findOne({ where: { email } });
  if (existing) {
    throw new ConflictError(
```

```
      'Пользователь с таким email уже существует');
  }
  const passwordHash = await hashPassword(password);
  const user = await User.create(
    { email, passwordHash, role: ROLES.USER },
    { fields: ['email', 'passwordHash', 'role'] });
  return user;
}
```

При входе пароль сравнивается с хешем, после чего выдаётся пара токенов: короткоживущий *access*"=токен (15 минут) возвращается в теле ответа, а *refresh*"=токен — в *httpOnly cookie*. *Middleware authenticate* проверяет формат заголовка *Authorization*, подпись и срок действия токена, а затем загружает пользователя из базы. Роль берётся из базы, а не из токена, поэтому понижение роли действует сразу:

```
async function authenticate(req, res, next) {
  const payload = decodeAccessToken(req,
    extractBearerToken(req));
  const user = await User.findByPk(Number(payload.sub));
  if (!user) {
    throw new UnauthorizedError('Пользователь не найден',
      'TOKEN_INVALID');
  }
  req.actor = { id: user.id, email: user.email,
    role: user.role, ip: req.ip };
  next();
}
```

2.7 Проверка регистрации и входа

Все запросы выполнялись к работающему серверу, ответы на рисунках получены от него без изменений; значения токенов и паролей сокращены или скрыты. Результат регистрации нового пользователя представлен на рисунке 2.2. Сервер вернул статус 201 и открытые поля пользователя, роль назначена автоматически.

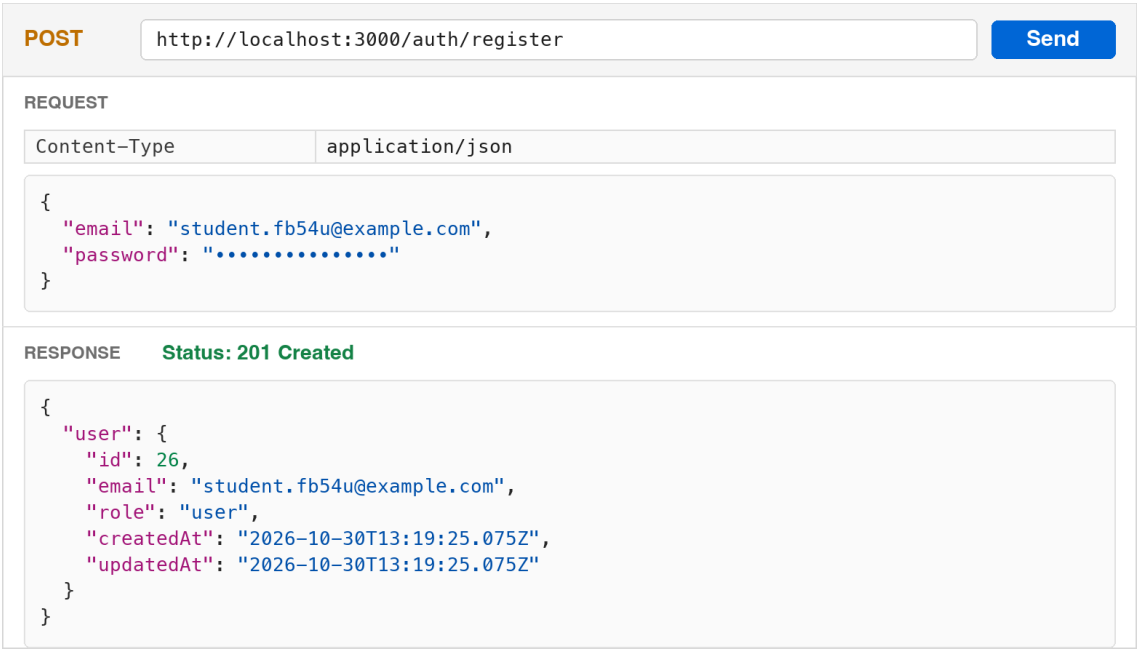


Рисунок 2.2 – Регистрация нового пользователя

Повторная регистрация с тем же *email* отклонена: ответ приведён на рисунке 2.3. Это подтверждает уникальность адреса.



Рисунок 2.3 – Повторная регистрация с занятым *email*

Запрос с некорректными данными показан на рисунке 2.4. Сервер вернул статус 400 с кодом *VALIDATION_ERROR* и списком ошибок по каждому полю.

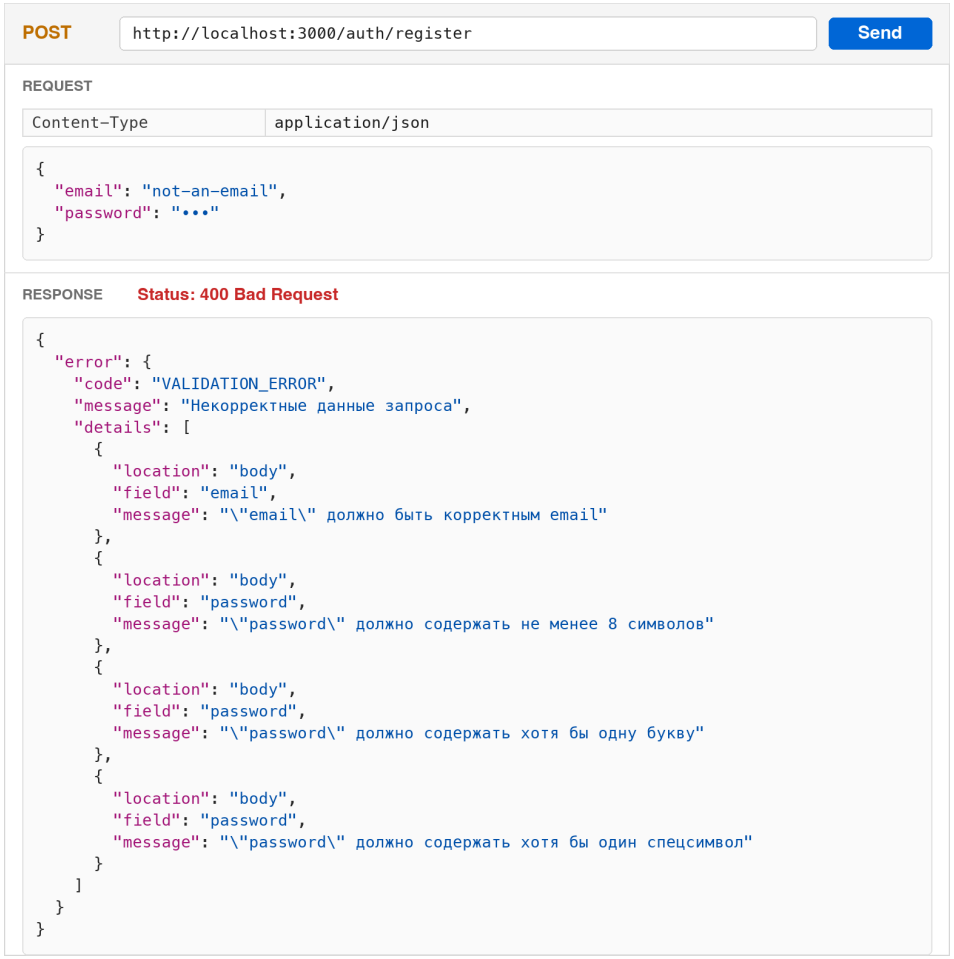


Рисунок 2.4 – Ошибки валидации при регистрации

Вход с верными данными представлен на рисунке 2.5. В ответе приходят данные пользователя и *JWT* для заголовка *Authorization*.

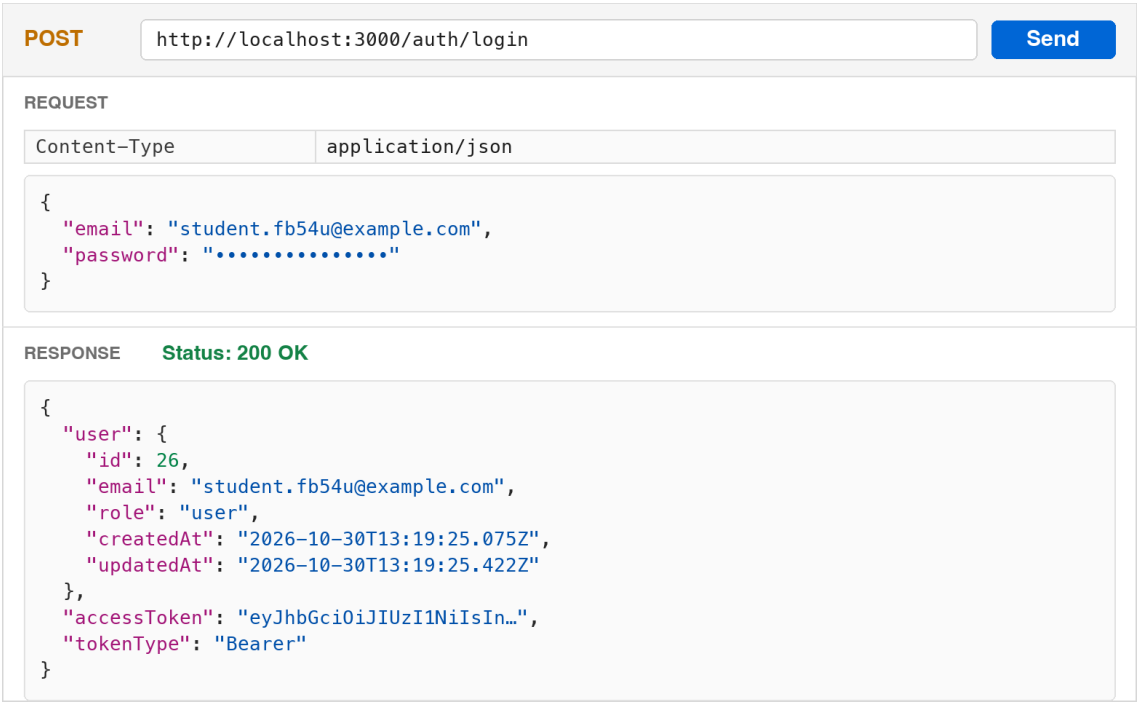


Рисунок 2.5 – Вход и получение *JWT*

Вход с неверным паролем завершается статусом 401 и общим сообщением, не выдающим существование учётной записи (рисунок 2.6).

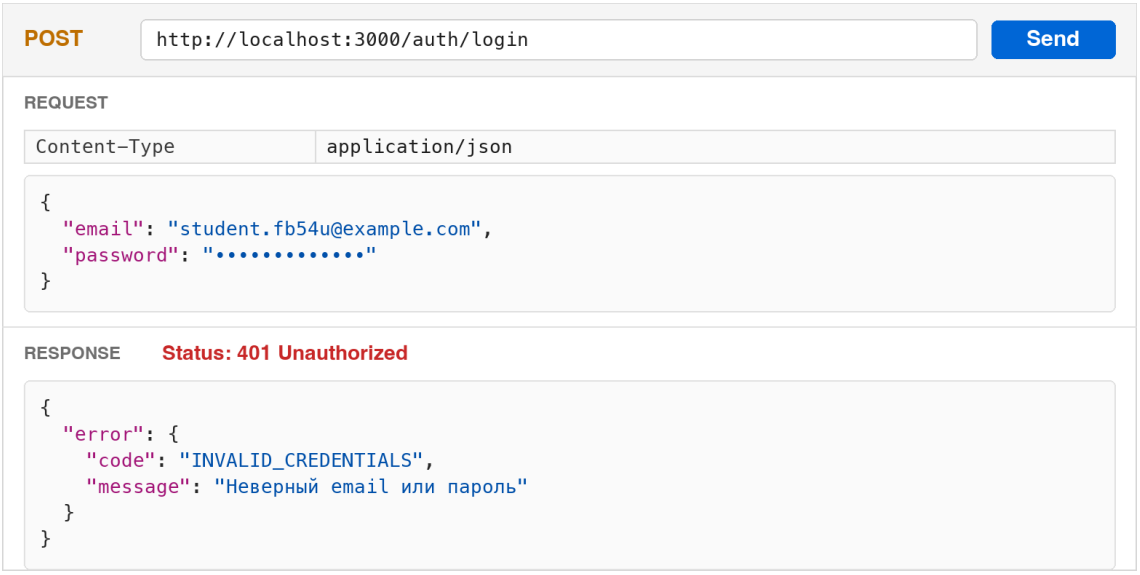


Рисунок 2.6 – Вход с неверным паролем

Доступ к защищённому маршруту без токена и с поддельным токеном показан на рисунках 2.7 и 2.8: в обоих случаях сервер возвращает статус 401 с разными кодами ошибки.

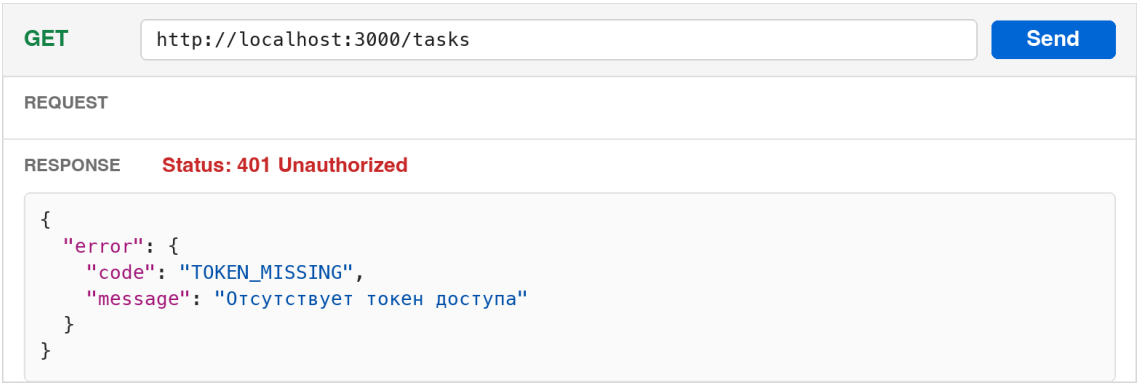


Рисунок 2.7 – Запрос к защищённому маршруту без токена

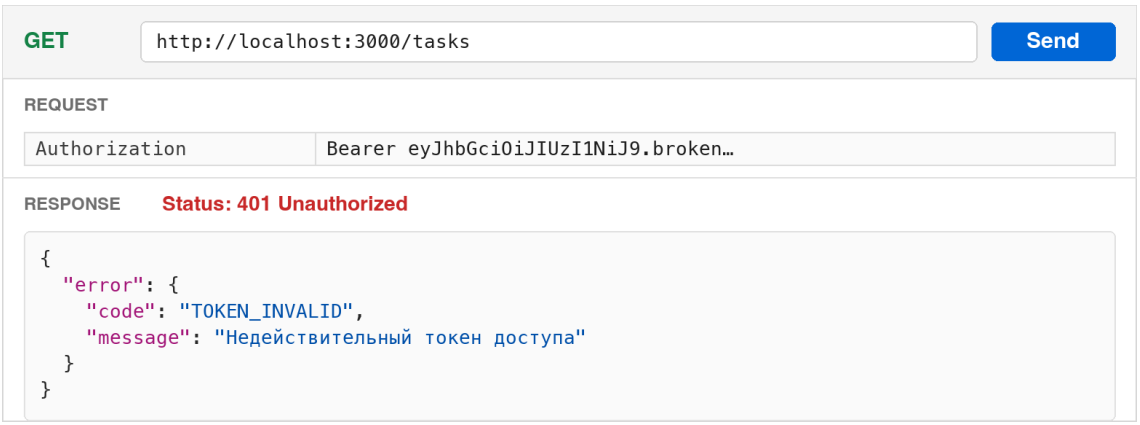


Рисунок 2.8 – Запрос с поддельным токеном

2.8 Проверка *CRUD*-операций

Создание задачи с токеном пользователя показано на рисунке 2.9. Сервер вернул статус 201, присвоил задаче идентификатор и владельца, а даты привёл к единому формату.

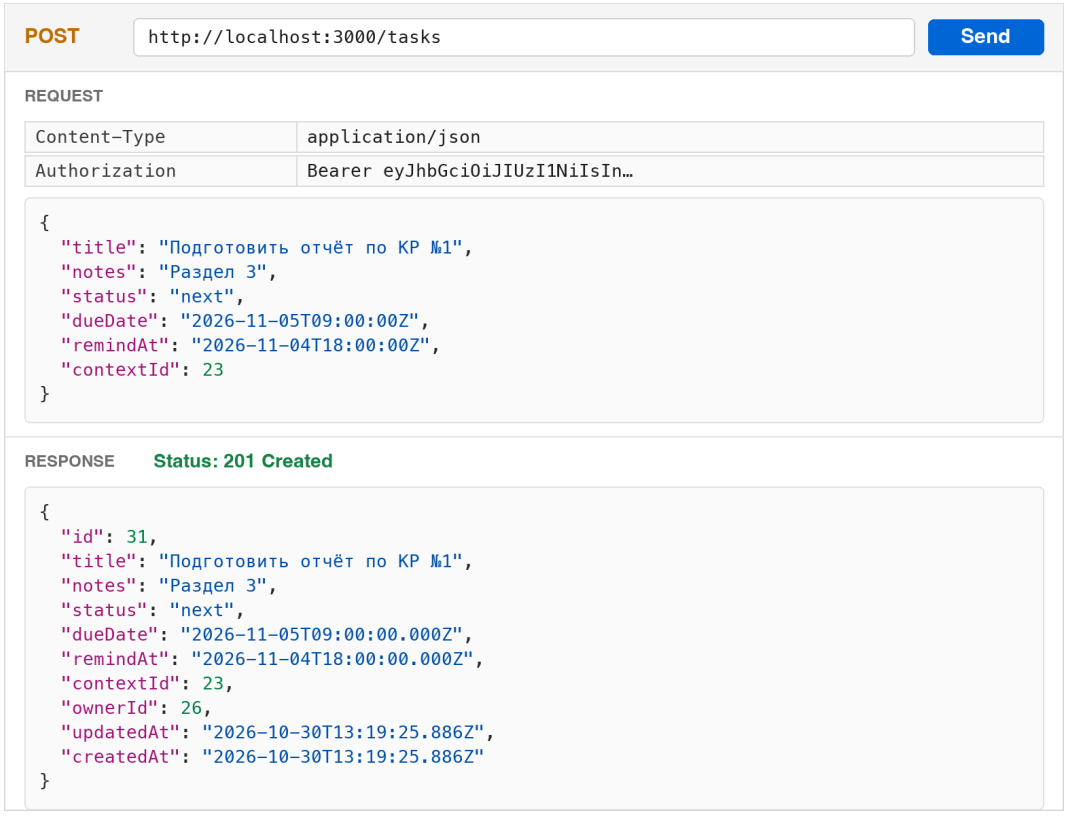


Рисунок 2.9 – Создание задачи

Получение списка с фильтрацией по статусу и контексту (дополнительный эндпоинт) приведено на рисунке 2.10. В ответ попадают только задачи, подходящие под оба условия, а вместе с ними — общее число записей и параметры страницы.

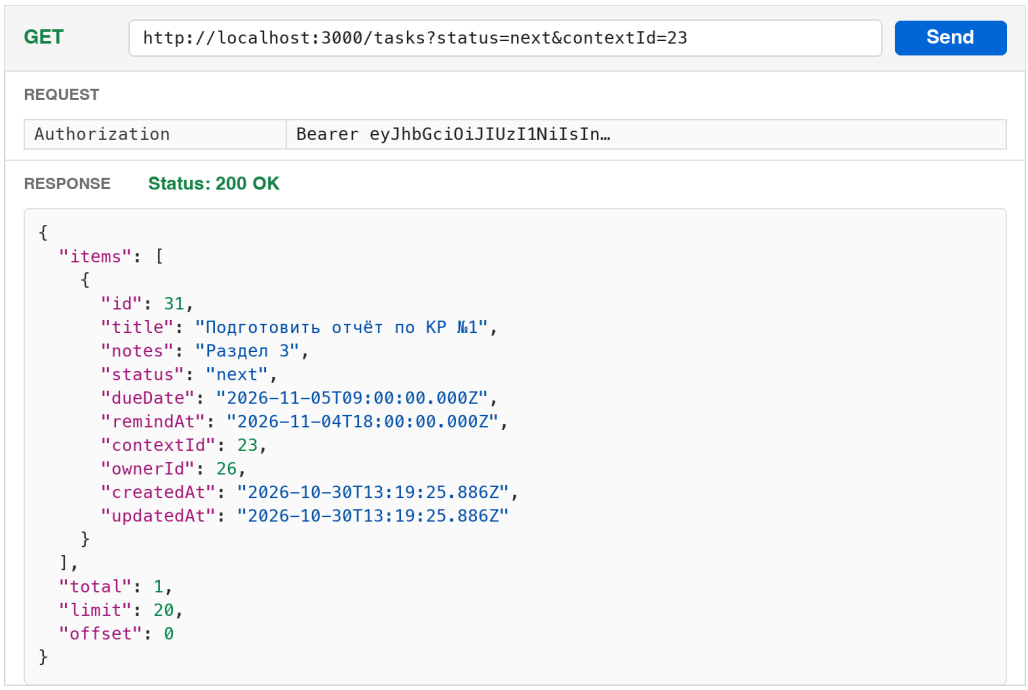


Рисунок 2.10 – Список задач с фильтрацией

Получение одной задачи по идентификатору представлено на рисунке 2.11.

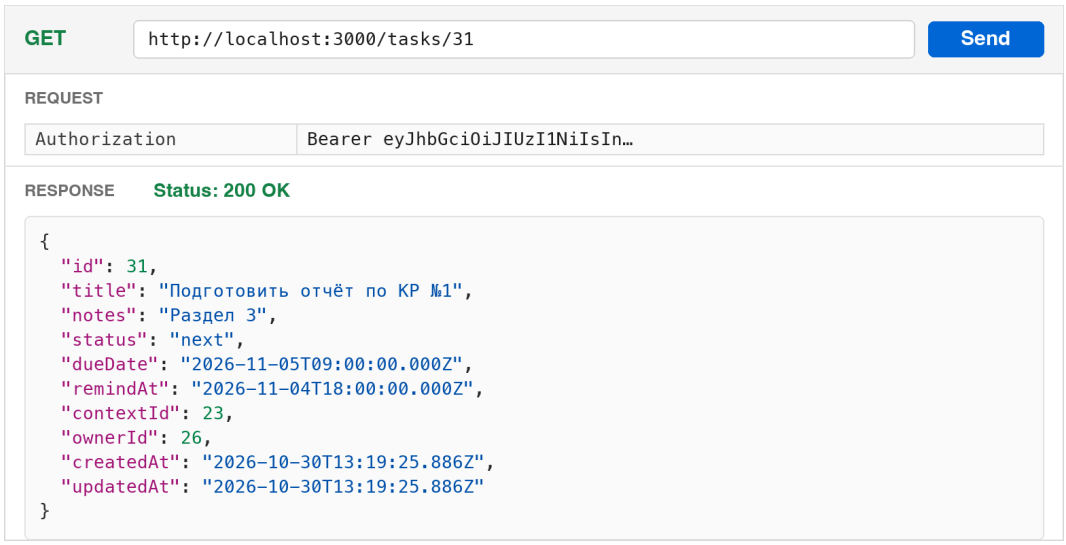


Рисунок 2.11 – Получение задачи по идентификатору

Полная замена задачи методом *PUT* показана на рисунке 2.12. Не переданные поля сброшены к значениям по умолчанию.

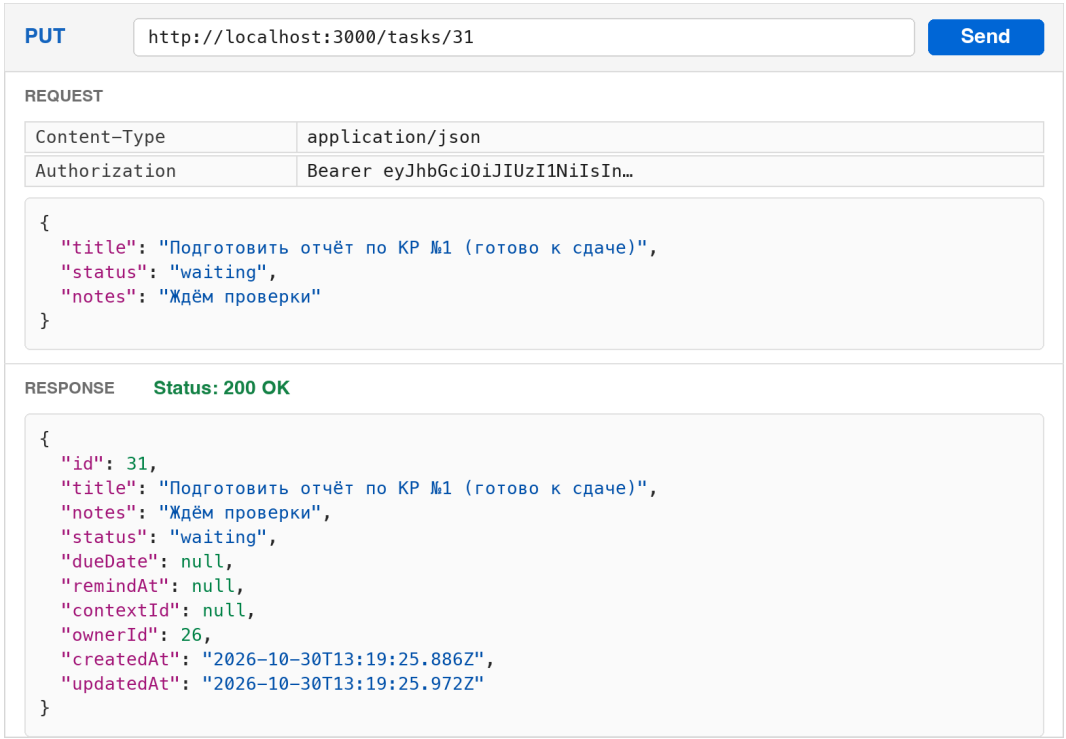


Рисунок 2.12 – Полная замена задачи методом *PUT*

Ошибки валидации при создании и при замене задачи приведены на рисунках 2.13 и 2.14: пустой заголовок и недопустимый статус, а также отсутствие обязательного поля *title* отклоняются статусом 400.

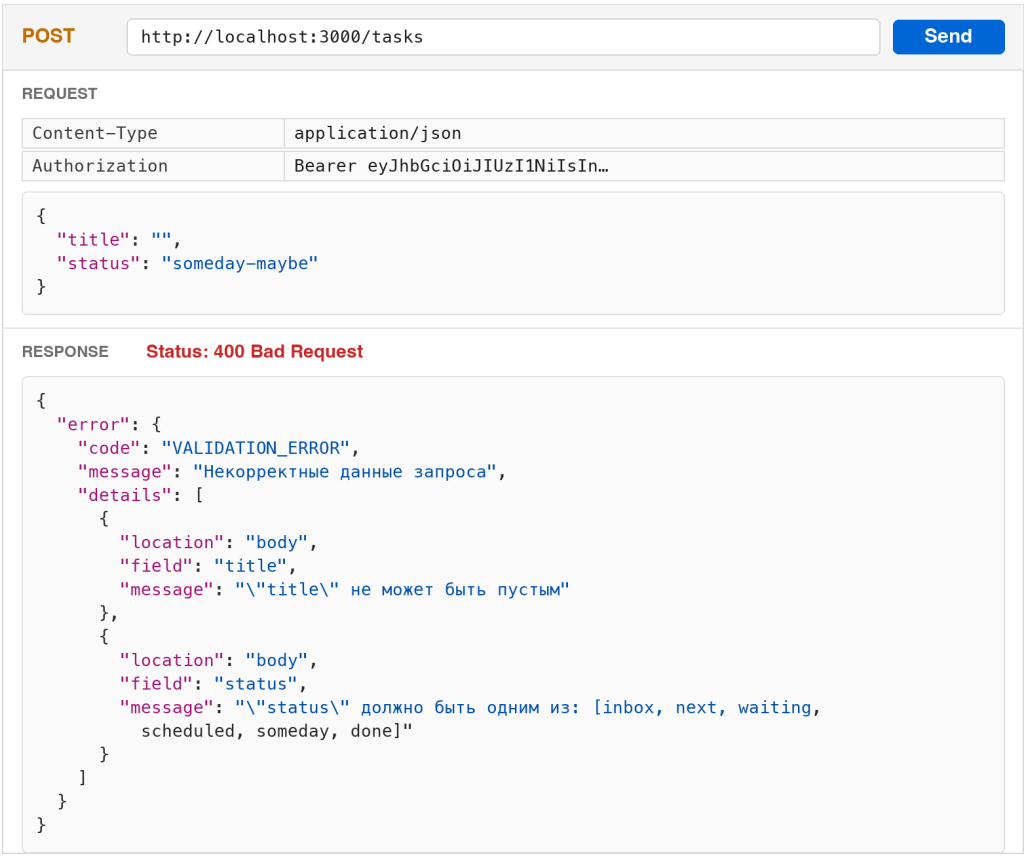


Рисунок 2.13 – Создание задачи с недопустимыми данными

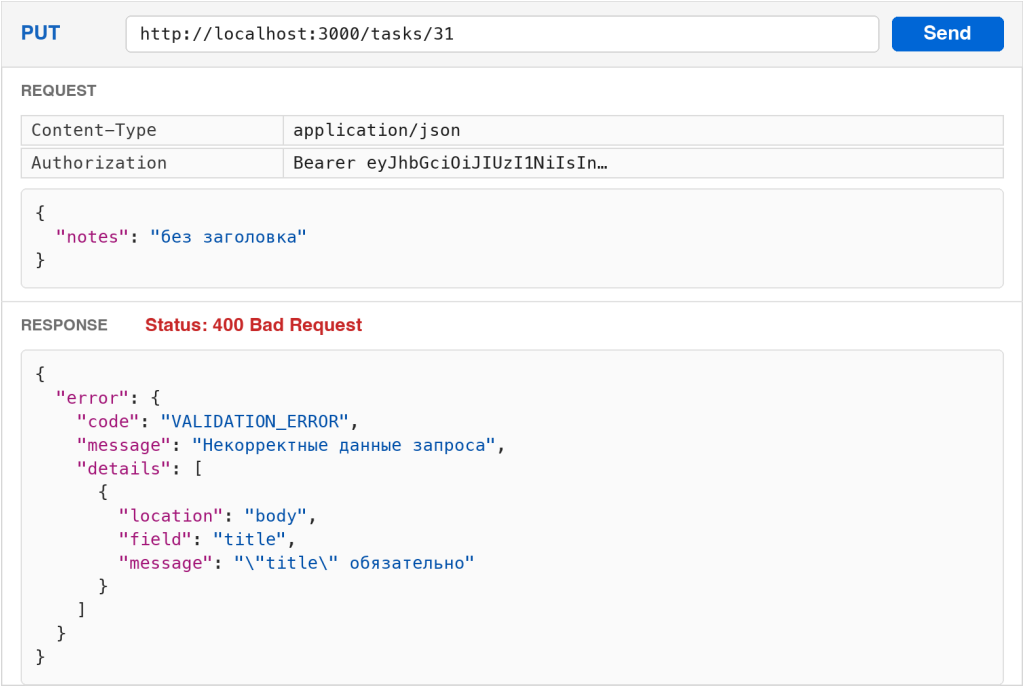


Рисунок 2.14 – Замена задачи без заголовка

Права на удаление проверены двумя запросами. Попытка удалить задачу от имени пользователя с ролью *user* показана на рисунке 2.15: сервер вернул статус 403 с кодом *ADMIN_ONLY*, даже несмотря на то что пользователь является владельцем.

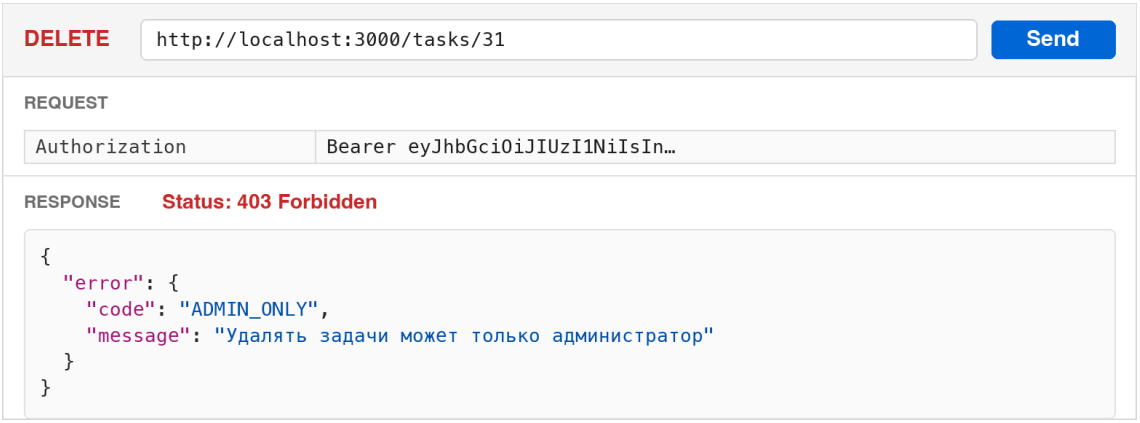


Рисунок 2.15 – Попытка удаления от имени *user*

Удаление той же задачи администратором представлено на рисунке 2.16: сервер вернул статус 204 без тела ответа.

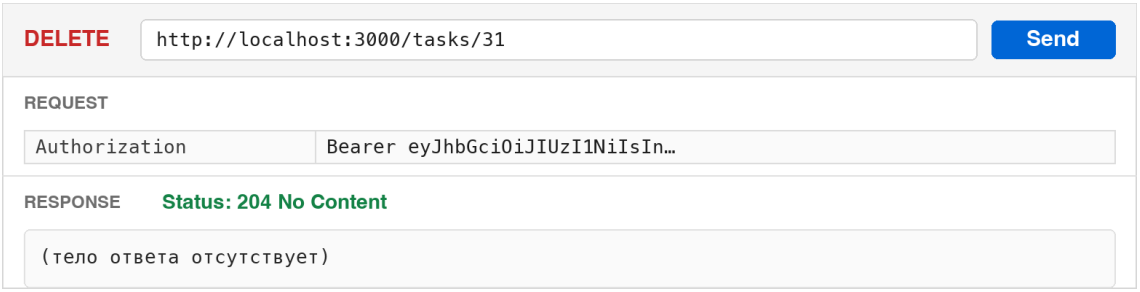


Рисунок 2.16 – Удаление задачи администратором

Повторное обращение к удалённой задаче завершилось статусом 404 (рисунок 2.17).



Рисунок 2.17 – Запрос удалённой задачи

2.9 Документация и автоматическое тестирование

API описан в файле *API.md* и в спецификации *OpenAPI 3.0*, которая отдаётся интерактивной страницей *Swagger UI* по адресу */api-docs*. Раздел задач из *Swagger UI* приведён на рисунке 2.18: видны все маршруты, включая *PUT* и *DELETE* с пометкой «только *admin*».

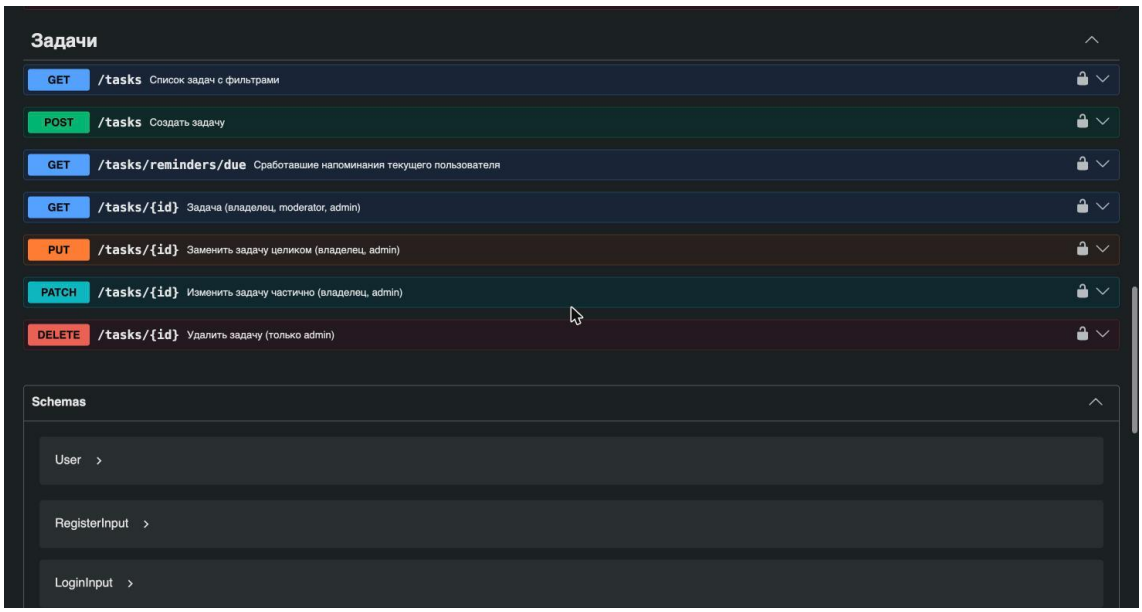


Рисунок 2.18 – Раздел задач в *Swagger UI*

Для повторяемой проверки подготовлена коллекция *Postman* из 129 запросов и 196 проверок: регистрация, вход, *CRUD*, ошибки валидации, доступ без токена, чужие ресурсы и роли. Прогон коллекции через *Newman* завершился без ошибок, результат показан на рисунке 2.19.

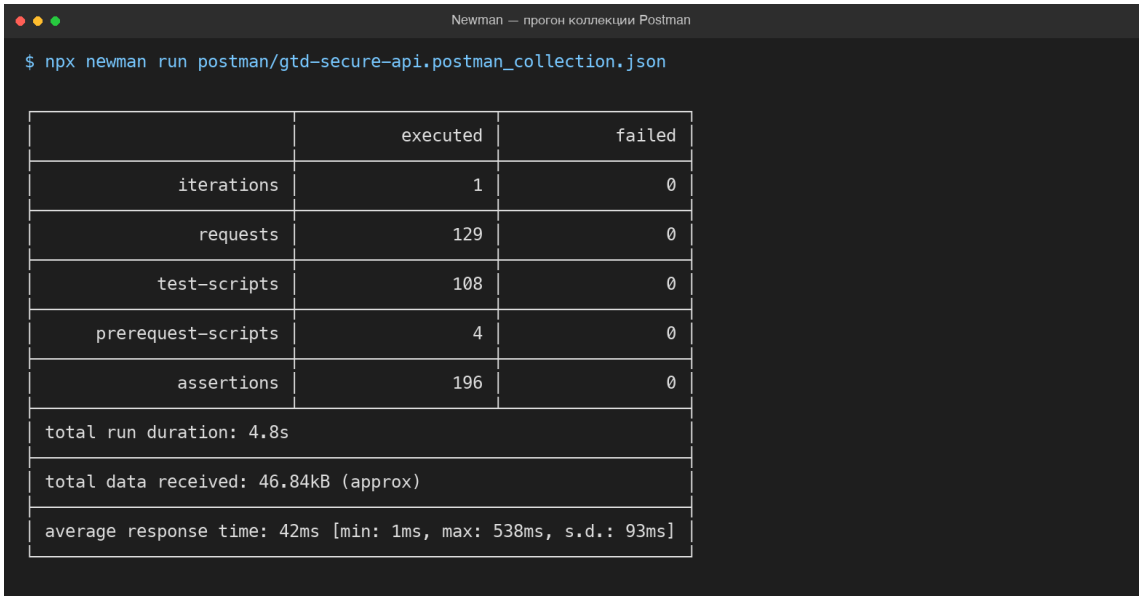


Рисунок 2.19 – Прогон коллекции *Postman* через *Newman*

3 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1 Выбрана сущность «задача» (*Task*): сервис планирования по методу *GTD* — тема курсового проекта, а задача является его центральным объектом. Сущность содержит заголовок, заметки, статус, срок, время напоминания, контекст и владельца.

2 Использованы *Node.js* и *Express 5* как привычный и быстрый стек для *REST API*, *PostgreSQL* и *Sequelize* для хранения связанных данных с проверкой целостности, *jsonwebtoken* и *bcrypt* для аутентификации, *Joi* для валидации, *helmet*, *cors* и *express-rate-limit* для защиты.

3 Пользователь регистрируется запросом *POST /auth/register*: пароль хешируется, запись сохраняется с ролью *user*. При входе *POST /auth/login* пароль сверяется с хешем, после чего выдаются *access*”=токен в теле ответа и *refresh*”=токен в *httpOnly cookie*. Защищённые маршруты проверяют заголовок *Authorization: Bearer*.

4 Пароли хешируются алгоритмом *bcrypt* с 12 раундами; соль генерируется автоматически и хранится в хеше. Это важно, потому что при утечке базы злоумышленник получает не пароли, а хеши, подбор которых требует много времени и ресурсов.

5 Реализованы роли *user*, *moderator* и *admin*. Пользователь создаёт задачи и работает со своими. Модератор дополнительно читает чужие задачи. Администратор дополнительно изменяет любые задачи, удаляет задачи и управляет пользователями. Удалять задачи может только администратор.

6 Перед созданием пользователя выполняется поиск по *email*, при совпадении возвращается статус 409. Дополнительно на столбец наложено ограничение *UNIQUE*, которое защищает от гонки двух одновременных регистраций. Адрес предварительно приводится к нижнему регистру.

7 *GET* — получение списка и отдельной задачи, *POST* — регистрация, вход и создание задач, *PUT* — полная замена задачи, *PATCH* — частичное изменение, *DELETE* — удаление задачи администратором.

8 Каждый маршрут имеет схему *Joi*. При ошибке возвращается статус 400 и объект с кодом *VALIDATION_ERROR* и списком нарушений. Пример ответа: *{ ”error”: { ”code”: ”VALIDATION_ERROR”, ”message”: ”Некорректные данные запроса”, ”details”: [{ ”location”: ”body”, ”field”: ”email”, ”message”: ”должно быть корректным email” }] } }*.

9 В коде нет сырого *SQL*: все запросы строятся методами *Sequelize*, которые передают значения как параметры, а не подставляют их в текст запроса. Идентификаторы дополнительно приводятся *Joi* к целым числам.

10 Применены *helmet* (строгая *CSP*, *nosniff*, запрет встраивания), ограничение частоты запросов (глобально 300 за 15 минут, для маршрутов */auth/**

— 20 неуспешных), *CORS* с белым списком источников, лимит тела запроса 10 КБ, приём только *application/json*, санитизация *HTML* в строках и блокировка учётной записи после пяти неудачных входов.

11 Использованы *Swagger UI* по спецификации *OpenAPI 3.0* и файл *API.md*. Спецификация машиночитаема, по ней можно сразу пробовать запросы, а *API.md* удобно читать в репозитории.

12 Использованы коллекция *Postman* из 129 запросов с 196 проверками, запуск через *Newman* из консоли и ручные запросы к работающему серверу, результаты которых приведены на рисунках.

13 Создание — 201 *Created*, успешное обновление — 200 *OK*, удаление — 204 *No Content*. Ошибочные случаи: 400, 401, 403, 404 и 409.

14 Секреты (*JWT_SECRET*, *JWT_REFRESH_SECRET*, строка подключения к базе) хранятся в файле *.env*, который не попадает в репозиторий. В репозитории лежит только *.env.example* с заглушками. При запуске сервер не принимает секреты короче 32 символов.

15 *CORS* (*Cross-Origin Resource Sharing*) — механизм, по которому браузер разрешает странице с одного источника обращаться к *API* другого источника. Сервер принимает запросы только с источников из переменной *CORS_ORIGINS* (адреса разработки клиента *http://localhost:5173* и *http://localhost:3000*), запросы с чужих источников отклоняются.

16 Планируется подключить клиент на *React*, добавить рассылку напоминаний по расписанию, повторяющиеся задачи, общий доступ к спискам и развёртывание на облачной платформе.

ЗАКЛЮЧЕНИЕ

В ходе контрольной работы разработан *REST API* сервиса планирования задач по методу *GTD* на *Node.js* и *Express*. Реализованы регистрация и вход с *JWT*, хранение данных в *PostgreSQL* через *Sequelize*, полный набор *CRUD*”=операций для сущности «задача» с правами по ролям: создание — любая аутентифицированная роль, обновление — автор или администратор, удаление — только администратор.

Все входные данные проверяются схемами *Joi*, ошибки возвращаются в едином формате с кодами, результаты проверок приведены в разделе 2. Добавлены фильтрация списка задач и выборка сработавших напоминаний, подготовлены документация в *Swagger UI*, файле *API.md* и коллекция *Postman*, прогон которой через *Newman* прошёл без ошибок.

Код размещён в репозитории *https://github.com/Alexandr-Obraztsov/gtd-task-service* в ветке *kr21*, коллекция *Postman* — по адресу *https://github.com/Alexandr-Obraztsov/gtd-task-service/blob/kr21/postman/gtd-secure-api.postman_collection.json*. Разработанный *API* служит серверной частью курсового проекта и будет дополнен клиентским приложением.